

13장. 제네릭

이것이 자바다

Contents

- 1절. 왜 제네릭을 사용해야 하는가?
- 2절. 제네릭 타입
- 3절. 멀티 타입 파라미터
- 4절. 제네릭 메소드
- 5절. 제한된 타입 파라미터
- 6절. 와일드카드 타입
- 7절. 제네릭 타입의 상속과 구현



1절. 왜 제네릭을 사용해야 하는가?

❖ 제네릭(Generic) 타입이란?

- ‘컴파일 단계’ 에서 ‘잘못된 타입 사용될 수 있는 문제’ 제거 가능
- 자바5부터 새로 추가 !
- 컬렉션, 람다식(함수적 인터페이스), 스트림, NIO에서 널리 사용
- 제네릭을 모르면 API 도큐먼트 해석 어려우므로 학습 필요

Class ArrayList<E>

```
default BiConsumer<T,U> andThen(BiConsumer<? super T,? super U> after)
```



1절. 왜 제네릭을 사용해야 하는가?

❖ 제네릭을 사용하는 코드의 이점

- 컴파일 시 강한 타입 체크 가능
  - 실행 시 타입 에러가 나는 것 방지
  - 컴파일 시에 미리 타입을 강하게 체크해서 에러 사전 방지
- 타입변환 제거 가능

```
List list = new ArrayList();
list.add("hello");
String str = (String) list.get(0);
```



```
List<String> list = new ArrayList<String>();
list.add("hello");
String str = list.get(0);
```



2절. 제네릭 타입

❖ 제네릭 타입이란?

- 타입을 파라미터로 가지는 클래스와 인터페이스
- 선언 시 클래스 또는 인터페이스 이름 뒤에 “<>” 부호 붙임
- “<>” 사이에는 타입 파라미터 위치

- 타입 파라미터
  - 일반적으로 대문자 알파벳 한 문자로 표현
  - 개발 코드에서는 타입 파라미터 자리에 구체적인 타입을 지정해야



2절. 제네릭 타입

❖ 제네릭 타입 사용 여부에 따른 비교

- 제네릭 타입을 사용하지 않은 경우
  - Object 타입 사용 → 빈번한 타입 변환 발생 → 프로그램 성능 저하

```
public class Box {
   private Object object;
   public void set(Object object) { this.object = object; }
   public Object get() { return object; }
}
```

```
Box box = new Box();
box.set("hello");                  //String 타입을 Object 타입으로 자동 타입 변환해서 저장
String str = (String) box.get();   //Object 타입을 String 타입으로 강제 타입 변환해서 얻음
```



2절. 제네릭 타입

❖ 제네릭 타입 사용 여부에 따른 비교

▪ 제네릭 타입 사용한 경우

- 클래스 선언할 때 타입 파라미터 사용
- 컴파일 시 타입 파라미터가 구체적인 클래스로 변경

```
Box<String> box = new Box<String>();
```

```
public class Box<String> {
    private String t;
    public void set(String t) { this.t = t; }
    public String get() { return t; }
}
```

```
Box<String> box = new Box<String>();
box.set("hello");
String str = box.get();
```

```
public class Box<T> {
    private T t;
    public T get() { return t; }
    public void set(T t) { this.t = t; }
}
```

```
Box<Integer> box = new Box<Integer>();
```

```
public class Box<Integer> {
    private Integer t;
    public void set(Integer t) { this.t = t; }
    public Integer get() { return t; }
}
```

```
Box<Integer> box = new Box<Integer>();
box.set(6);
int value = box.get();
```

3절. 멀티 타입 파라미터

❖ 제네릭 타입은 두 개 이상의 타입 파라미터 사용 가능

- 각 타입 파라미터는 콤마로 구분
  - Ex) class<K, V, …> { … }
  - interface<K, V, …> { … }

```
public class Product<T, M> {
    private T kind;
    private M model;

    public T getKind() { return this.kind; }
    public M getModel() { return this.model; }

    public void setKind(T kind) { this.kind = kind; }
    public void setModel(M model) { this.model = model; }
}
```

```
Product<Tv, String> product = new Product<Tv, String>();
```

- 자바 7부터는 다이아몬드 연산자 사용해 간단히 작성과 사용 가능

```
Product<Tv, String> product = new Product<>();
```



4절. 제네릭 메소드

❖ 제네릭 메소드

- 매개변수 타입과 리턴 타입으로 타입 파라미터를 갖는 메소드
- 제네릭 메소드 선언 방법
  - 리턴 타입 앞에 “<>” 기호를 추가하고 타입 파라미터 기술
  - 타입 파라미터를 리턴 타입과 매개변수에 사용

```
public <타입파라미터,...> 리턴타입 메소드명(매개변수,...) { ... }
```

```
public <T> Box<T> boxing(T t) { ... }
```

- 제네릭 메소드 호출하는 두 가지 방법 (p.661~664)

```
리턴타입 변수 = <구체적타입> 메소드명(매개값);      //명시적으로 구체적 타입 지정
리턴타입 변수 = 메소드명(매개값);                   //매개값을 보고 구체적 타입을 추정
```

```
Box<Integer> box = <Integer>boxing(100);      //타입 파라미터를 명시적으로 Integer 로 지정
Box<Integer> box = boxing(100);               //타입 파라미터를 Integer 으로 추정
```

5절. 제한된 타입 파라미터

❖ 타입 파라미터에 지정되는 구체적인 타입 제한할 필요

- 상속 및 구현 관계 이용해 타입 제한

```
public <T extends 상위타입> 리턴타입 메소드(매개변수, ...) { ... }
```

  - 상위 타입은 클래스 뿐만 아니라 인터페이스도 가능

- 타입 파라미터를 대체할 구체적인 타입
  - 상위타입이거나 하위 또는 구현 클래스만 지정 가능
  - 사용시 주의할 점은 p.664~665 참조



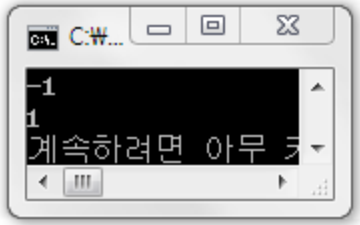
BoundedTypeParameterExample.java_제네릭 메소드 호출

```java
public class BoundedTypeParameterExample {
    public static void main(String[] args) {
        //String str = Util.compare("a", "b"); (x)

        int result1 = Util.compare(10, 20);
        System.out.println(result1);

        int result2 = Util.compare(4.5, 3);
        System.out.println(result2);
    }
}

```



Util.java_제네릭 메소드

```java
public class Util {
    public static <T extends Number> int compare(T t1, T t2) {
        double v1 = t1.doubleValue();
        //System.out.println(t1.getClass().getName());
        double v2 = t2.doubleValue();
        //System.out.println(t2.getClass().getName());
        return Double.compare(v1, v2);
    }
}

```

6절. 와일드카드 타입

❖ 와일드카드 타입의 세가지 형태

- 제네릭타입<?> : Unbounded Wildcards (제한없음)
  타입 파라미터를 대치하는 구체적인 타입으로 모든 클래스나 인터페이스 타입이 올 수 있다.

- 제네릭타입<? extends 상위타입> : Upper Bounded Wildcards (상위 클래스 제한)
  타입 파라미터를 대치하는 구체적인 타입으로 상위 타입이나 하위 타입만 올 수 있다.

- 제네릭타입<? super 하위타입> : Lower Bounded Wildcards (하위 클래스 제한)
  타입 파라미터를 대치하는 구체적인 타입으로 하위 타입이나 상위 타입이 올 수 있다.



WildCardExample.java_와일드카드 타입 매개 변수1/2

```
1  import java.util.Arrays;
2
3  public class WildCardExample {
4      public static void registerCourse(Course<?> course) {
5          System.out.println(course.getName() + " 수강생: " +
6                  Arrays.toString(course.getStudents()));
7      }
8
9      public static void registerCourseStudent(Course<? extends Student> course) {
10         System.out.println(course.getName() + " 수강생: " +
11                 Arrays.toString(course.getStudents()) );
12     }
13
14     public static void registerCourseWorker(Course<? super Worker> course) {
15         System.out.println(course.getName() + " 수강생: " +
16                 Arrays.toString(course.getStudents()));
17     }
18
19     public static void main(String[] args) {
20         Course<Person> personCourse = new Course<Person>("일반인과정", 5);
21             personCourse.add(new Person("일반인"));
22             personCourse.add(new Worker("직장인"));
23             personCourse.add(new Student("학생"));
24             personCourse.add(new HighStudent("고등학생"));
25
26         Course<Worker> workerCourse = new Course<Worker>("직장인과정", 5);
27             workerCourse.add(new Worker("직장인"));
28
```

WildCardExample.java_와일드카드 타입 매개 변수2/2

```
28
29          Course<Student> studentCourse = new Course<Student>("학생과정", 5);
30              studentCourse.add(new Student("학생"));
31              studentCourse.add(new HighStudent("고등학생"));
32
33          Course<HighStudent> highStudentCourse = new Course<HighStudent>("고등학생과정", 5);
34              highStudentCourse.add(new HighStudent("고등학생"));
35
36          registerCourse(personCourse);
37          registerCourse(workerCourse);
38          registerCourse(studentCourse);
39          registerCourse(highStudentCourse);
40          System.out.println();
41
42          //registerCourseStudent(personCourse);
43          //registerCourseStudent(workerCourse);
44          registerCourseStudent(studentCourse);
45          registerCourseStudent(highStudentCourse);
46          System.out.println();
47
48          registerCourseWorker(personCourse);
49          registerCourseWorker(workerCourse);
50          //registerCourseWorker(studentCourse);          (x)
51          //registerCourseWorker(highStudentCourse);      (x)
52      }
53  }
```

```
일반인과정 수강생: [일반인, 직장인, 학생, 고등학생, null]
직장인과정 수강생: [직장인, null, null, null, null]
학생과정 수강생: [학생, 고등학생, null, null, null]
고등학생과정 수강생: [고등학생, null, null, null, null]

학생과정 수강생: [학생, 고등학생, null, null, null]
고등학생과정 수강생: [고등학생, null, null, null, null]

일반인과정 수강생: [일반인, 직장인, 학생, 고등학생, null]
직장인과정 수강생: [직장인, null, null, null, null]
계속하려면 아무 키나 누르십시오 . . .
```

관련 클래스

- Course.java
- HighStudent.java
- Person.java
- Student.java
- WildCardExample.java
- Worker.java

Course.java_제네릭 타입

```
1  public class Course<T> {
2      private String name;
3      private T[] students;
4
5      public Course(String name, int capacity) {
6          this.name = name;
7          students = (T[]) (new Object[capacity]);
8      }
9
10     public String getName() { return name; }
11     public T[] getStudents() { return students; }
12     public void add(T t) {
13         for(int i=0; i<students.length; i++) {
14             if(students[i] == null) {
15                 students[i] = t;
16                 break;
17             }
18         }
19     }
20 }
```



HighStudent_Person_Student_Worker class

```
1  public class HighStudent extends Student {
2      public HighStudent(String name) {
3          super(name);
4      }
5  }
6
7  class Person {
8      private String name;
9
10     public Person(String name) {
11         this.name = name;
12     }
13
14     public String getName() { return name; }
15     public String toString() { return name; }
16 }
17
18 class Student extends Person {
19     public Student(String name) {
20         super(name);
21     }
22 }
23
24 class Worker extends Person {
25     public Worker(String name) {
26         super(name);
27     }
28 }
```



7절. 제네릭 타입의 상속과 구현

❖ 제네릭 타입을 부모 클래스로 사용할 경우

- 타입 파라미터는 자식 클래스에도 기술해야 !!!

```
public class ChildProduct<T, M> extends Product<T, M> { ... }
```

- 추가적인 타입 파라미터 가질 수 있음

```
public class ChildProduct<T, M, C> extends Product<T, M> { ... }
```

❖ 제네릭 인터페이스를 구현할 경우

- 제네릭 인터페이스를 구현한 클래스도 제네릭 타입
  - 예제 (p.670~671)를 통해 개념 확실히 이해할 것



Product.java_부모 제네릭 클래스

```java
public class Product<T, M> {
    private T kind;
    private M model;

    public T getKind() { return this.kind; }
    public M getModel() { return this.model; }

    public void setKind(T kind) { this.kind = kind; }
    public void setModel(M model) { this.model = model; }
}

class Tv {}
```

ChildProduct.java_자식 제네릭 클래스

```java
public class ChildProduct<T, M, C> extends Product<T, M> {
        private C company;
        public C getCompany() { return this.company; }
        public void setCompany(C company) { this.company = company; }
}

```

Storage.java_제네릭 인터페이스

```java
public interface Storage<T> {
    public void add(T item, int index);
    public T get(int index);
}

```



StorageImpl.java_제네릭 구현 클래스

```
1  public class Course<T> {
2      private String name;
3      private T[] students;
4
5      public Course(String name, int capacity) {
6          this.name = name;
7          students = (T[]) (new Object[capacity]);
8      }
9
10     public String getName() { return name; }
11     public T[] getStudents() { return students; }
12     public void add(T t) {
13         for(int i=0; i<students.length; i++) {
14             if(students[i] == null) {
15                 students[i] = t;
16                 break;
17             }
18         }
19     }
20 }
```



ChildProductAndStorageExample.java_제네릭 타입 사용 클래스

```
1  public class ChildProductAndStorageExample {
2      public static void main(String[] args) {
3          ChildProduct<Tv, String, String> product = new ChildProduct<>();
4          product.setKind(new Tv());
5          product.setModel("SmartTV");
6          product.setCompany("Samsung");
7
8          Storage<Tv> storage = new StorageImpl<Tv>(100);
9          storage.add(new Tv(), 0);
10         Tv tv = storage.get(0);
11     }
12 }
13
```

관련 클래스

- ChildProduct.java
- ChildProductAndStorageExample.java
- Product.java
- Storage.java
- StorageImpl.java